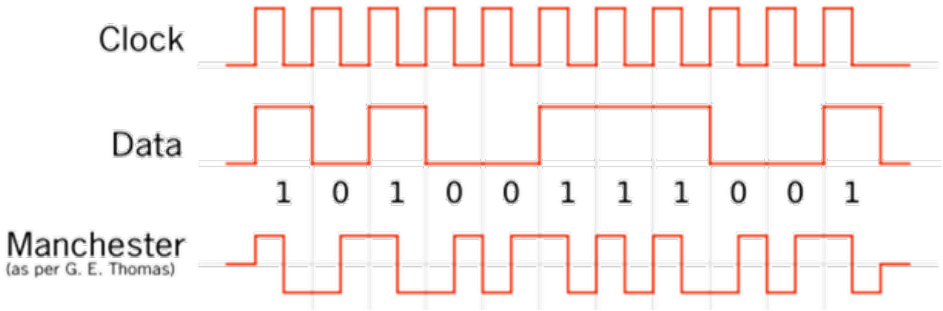
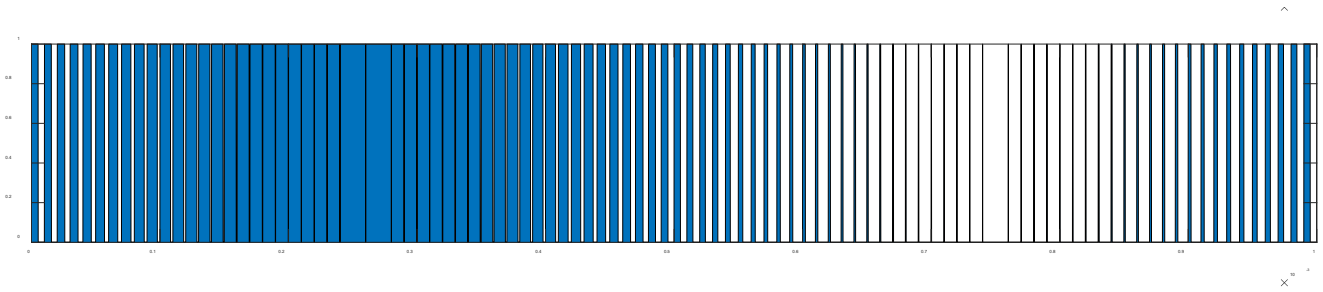
Lab 4: PWM Audio and Addressable LEDs

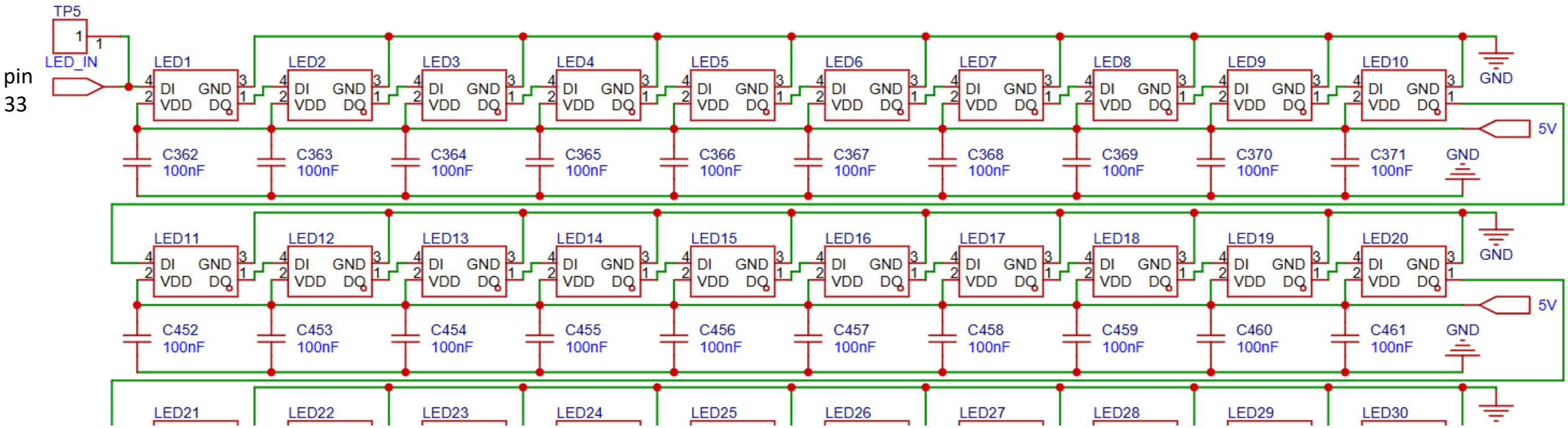
- In this lab you will write low-level drivers for two peripherals:
  - The ESP32 RMT (Remote Control Transceiver)
    - Generates a waveform intended for infrared remote control signals
    - These generally use Manchester code:



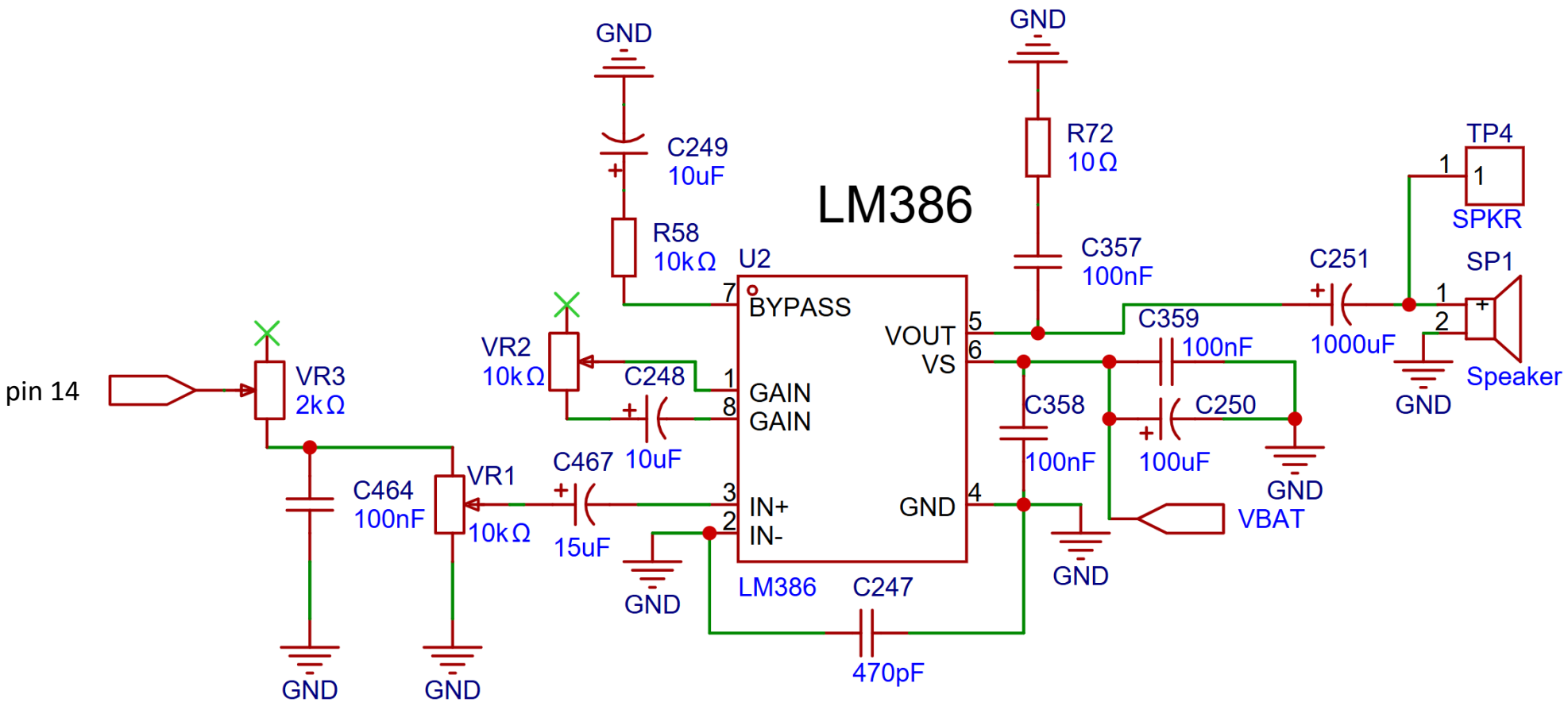
  - The ESP32 LEDC (LED Controller)
    - Intended for LEDs
    - Generates a PWM signal:



WS2812 Array Schematic

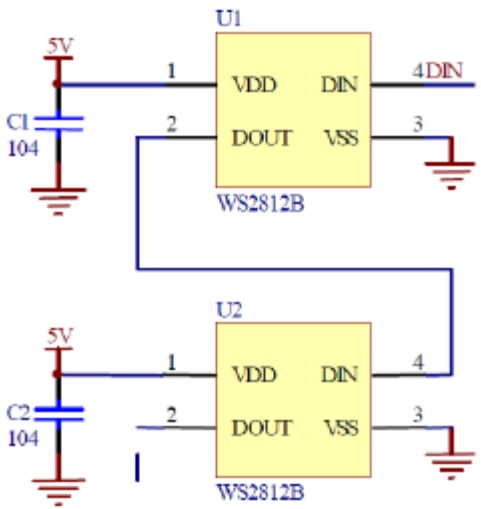


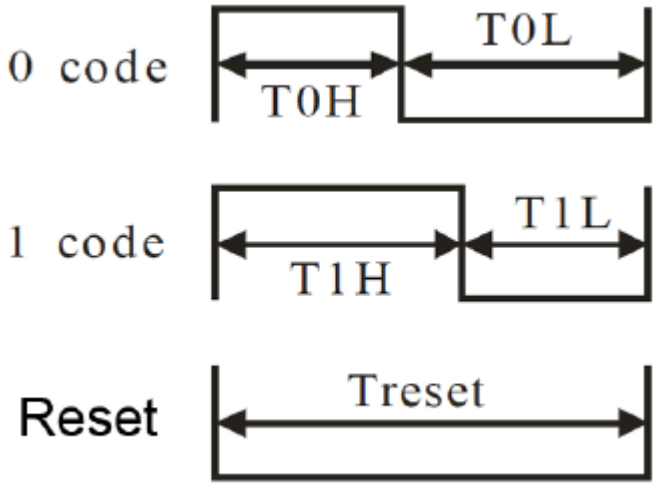
Audio Circuit Schematic



WS2812 LEDs

- Individually-addressed LED
- Asynchronous, one-input DIN, one-output DOUT





T0H	400 ns
T0L	850 ns
T1H	800 ns
T1L	450 ns
reset	50 us

Initializing RMT

Register 17.4. SYSTEM_PERIP_CLK_EN0_REG (0x0018)

31	30	29	28	27 … 25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)	SYSTEM_ADC2_ARB_CLK_EN	SYSTEM_SYSTIMER_CLK_EN	SYSTEM_APB_SARADC_CLK_EN	(reserved)	SYSTEM_UART_MEM_CLK_EN	SYSTEM_USB_CLK_EN	(reserved)	SYSTEM_I2S1_CLK_EN	SYSTEM_PWM1_CLK_EN	SYSTEM_CAN_CLK_EN	SYSTEM_I2C_EXT1_CLK_EN	SYSTEM_PWM0_CLK_EN	SYSTEM_SPI3_CLK_EN	SYSTEM_TIMERGROUP1_CLK_EN	(reserved)	SYSTEM_TIMERGROUP_CLK_EN	(reserved)	SYSTEM_LEDC_CLK_EN	SYSTEM_PCNT_CLK_EN	SYSTEM_RMT_CLK_EN	SYSTEM_UHCI0_CLK_EN	SYSTEM_I2C_EXT0_CLK_EN	SYSTEM_SPI2_CLK_EN	SYSTEM_UART1_CLK_EN	SYSTEM_I2S0_CLK_EN	(reserved)	SYSTEM_UART_CLK_EN	SYSTEM_SPI01_CLK_EN	(reserved)		
0	1	1	0	0 0 0	1	1	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0	0	1	1	0	0	1	1	0	Reset

Register 17.6. SYSTEM_PERIP_RST_EN0_REG (0x0020)

31	30	29	28	27 … 25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
(reserved)	SYSTEM_ADC2_ARB_RST	SYSTEM_SYSTIMER_RST	SYSTEM_APB_SARADC_RST	(reserved)	SYSTEM_UART_MEM_RST	SYSTEM_USB_RST	(reserved)	SYSTEM_I2S1_RST	SYSTEM_PWM1_RST	SYSTEM_CAN_RST	SYSTEM_I2C_EXT1_RST	SYSTEM_PWM0_RST	SYSTEM_SPI3_RST	SYSTEM_TIMERGROUP1_RST	(reserved)	SYSTEM_TIMERGROUP_RST	(reserved)	SYSTEM_LEDC_RST	SYSTEM_PCNT_RST	SYSTEM_RMT_RST	SYSTEM_UHCI0_RST	SYSTEM_I2C_EXT0_RST	SYSTEM_SPI2_RST	SYSTEM_UART1_RST	SYSTEM_I2S0_RST	(reserved)	SYSTEM_UART_RST	SYSTEM_SPI01_RST	(reserved)		
0	0	0	0	0 0 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Initializing RMT

Register 37.7. RMT_SYS_CONF_REG (0x00C0)

RMT_CLK_EN	(reserved)	RMT_SCLK_ACTIVE	RMT_SCLK_SEL	RMT_SCLK_DIV_B	RMT_SCLK_DIV_A	RMT_SCLK_DIV_NUM	RMT_MEM_FORCE_PU	RMT_MEM_FORCE_PD	RMT_MEM_CLK_FORCE_ON	RMT_APB_FIFO_MASK	
31	30 27	26	25 24	23 18	17 12	11 4	3	2	1	0	
0	0 0 0 0	1	0x1	0x0	0x0	0x1	0	0	0	0	Reset

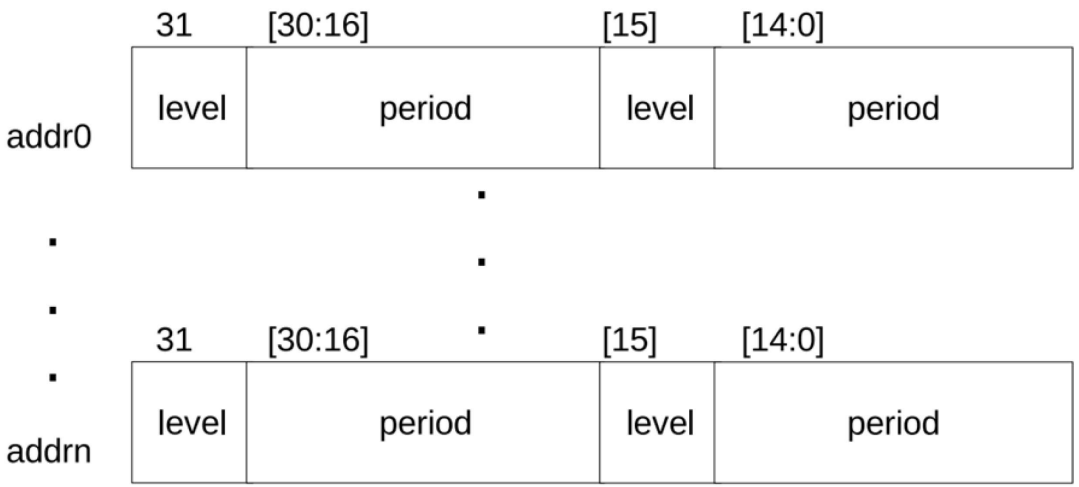
Set to 0 for 80 MHz clock
1/80MHz = 12.5 ns
850 ns / 12.5 ns = 68

Initializing RMT

Register 37.3. RMT_CH*n*CONF0_REG (*n*: 0-3) (0x0020+0x4n*)**

(reserved)	RMT_DMA_ACCESS_EN_CH3(reserved for n:0-2)	RMT_CONF_UPDATE_CHn	(reserved)	RMT_CARRIER_OUT_LV_CHn	RMT_CARRIER_EN_CHn	RMT_CARRIER_EFF_EN_CHn	RMT_MEM_SIZE_CHn	RMT_DIV_CNT_CHn	RMT_TX_STOP_CHn	RMT_IDLE_OUT_EN_CHn	RMT_IDLE_OUT_LV_CHn	RMT_MEM_TX_WRAP_EN_CHn	RMT_TX_CONTI_MODE_CHn	RMT_APB_MEM_RST_CHn	RMT_MEM_RD_RST_CHn	RMT_TX_START_CHn	
31 … 26	25	24	23	22	21	20	19 … 16	15 … 8	7	6	5	4	3	2	1	0	
0 0 0 0 0 0 0	0	0	0	1	1	1	0x1	0x2	0	0	0	0	0	0	0	0	Reset

RMT RAM



- RMT RAM holds a waveform as:
  - A sequence of entries
  - Each entry contains a four values
  - Zero terminated
- To initiate:
  - Write 24 entries + zero termination
  - Set then clear the RMT_MEM_RD_RST bit
  - Set RMT_TX_START
  - Wait for RMT_CH0_TX_END_INT_RAW to become set
  - Set the RMT_CH0_TX_END_INT_CLR bit

Using RMT

Register 37.3. RMT_CH*n*CONF0_REG (*n*: 0-3) (0x0020+0x4*n*)

(reserved)	RMT_DMA_ACCESS_EN_CH3(reserved for n:0-2)	RMT_CONF_UPDATE_CHn	(reserved)	RMT_CARRIER_OUT_LV_CHn	RMT_CARRIER_EN_CHn	RMT_CARRIER_EFF_EN_CHn	RMT_MEM_SIZE_CHn	RMT_DIV_CNT_CHn	RMT_TX_STOP_CHn	RMT_IDLE_OUT_EN_CHn	RMT_IDLE_OUT_LV_CHn	RMT_MEM_TX_WRAP_EN_CHn	RMT_TX_CONTI_MODE_CHn	RMT_APB_MEM_RST_CHn	RMT_MEM_RD_RST_CHn	RMT_TX_START_CHn	
31 … 26	25	24	23	22	21	20	19 … 16	15 … 8	7	6	5	4	3	2	1	0	
0 0 0 0 0 0 0	0	0	0	1	1	1	0x1	0x2	0	0	0	0	0	0	0	0	Reset

Using RMT

Register 15.3. RMT_INT_RAW_REG (0x00A0)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
RMT_CH7_TX_THR_EVENT_INT_RAW	RMT_CH6_TX_THR_EVENT_INT_RAW	RMT_CH5_TX_THR_EVENT_INT_RAW	RMT_CH4_TX_THR_EVENT_INT_RAW	RMT_CH3_TX_THR_EVENT_INT_RAW	RMT_CH2_TX_THR_EVENT_INT_RAW	RMT_CH1_TX_THR_EVENT_INT_RAW	RMT_CH0_TX_THR_EVENT_INT_RAW	RMT_CH7_ERR_INT_RAW	RMT_CH7_RX_END_INT_RAW	RMT_CH7_TX_END_INT_RAW	RMT_CH6_ERR_INT_RAW	RMT_CH6_RX_END_INT_RAW	RMT_CH6_TX_END_INT_RAW	RMT_CH5_ERR_INT_RAW	RMT_CH5_RX_END_INT_RAW	RMT_CH5_TX_END_INT_RAW	RMT_CH4_ERR_INT_RAW	RMT_CH4_RX_END_INT_RAW	RMT_CH4_TX_END_INT_RAW	RMT_CH3_ERR_INT_RAW	RMT_CH3_RX_END_INT_RAW	RMT_CH3_TX_END_INT_RAW	RMT_CH2_ERR_INT_RAW	RMT_CH2_RX_END_INT_RAW	RMT_CH2_TX_END_INT_RAW	RMT_CH1_ERR_INT_RAW	RMT_CH1_RX_END_INT_RAW	RMT_CH1_TX_END_INT_RAW	RMT_CH0_ERR_INT_RAW	RMT_CH0_RX_END_INT_RAW	RMT_CH0_TX_END_INT_RAW	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Register 15.6. RMT_INT_CLR_REG (0x00AC)

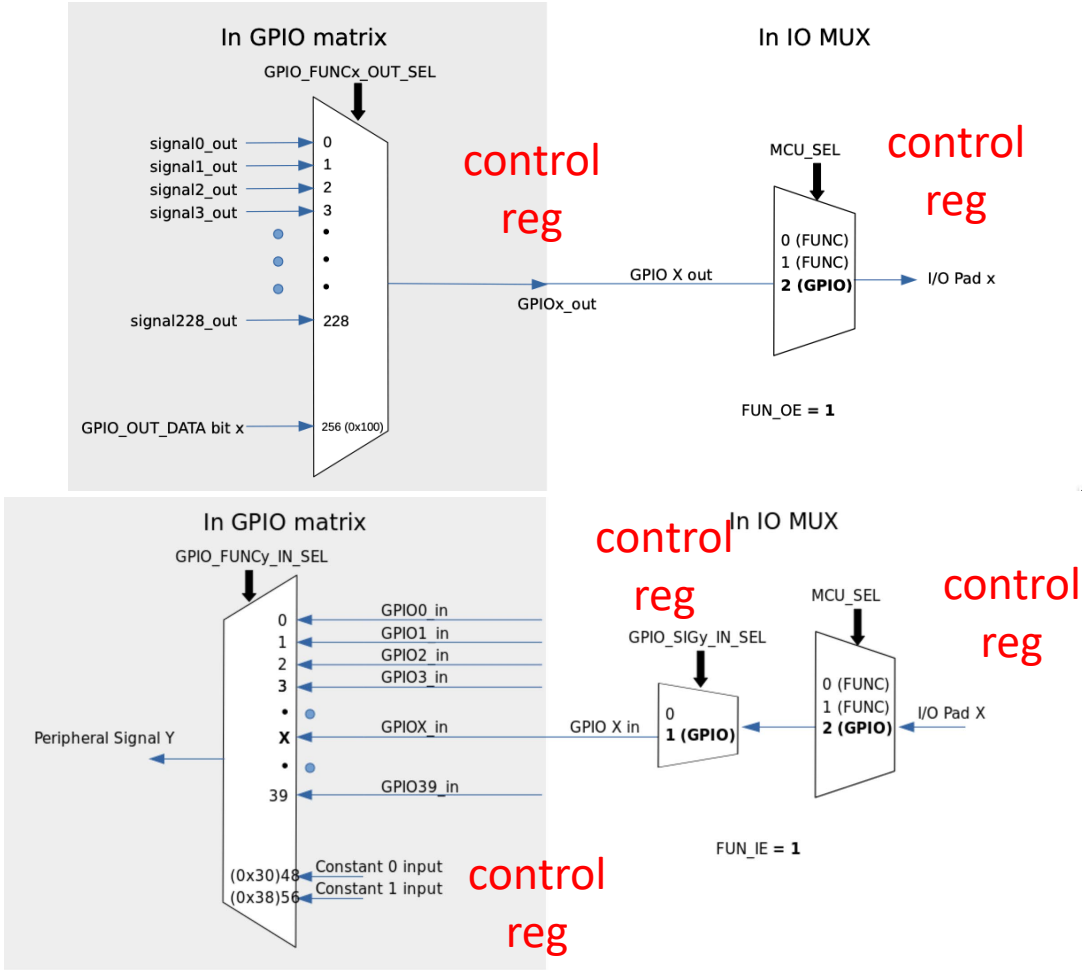
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
RMT_CH7_TX_THR_EVENT_INT_CLR	RMT_CH6_TX_THR_EVENT_INT_CLR	RMT_CH5_TX_THR_EVENT_INT_CLR	RMT_CH4_TX_THR_EVENT_INT_CLR	RMT_CH3_TX_THR_EVENT_INT_CLR	RMT_CH2_TX_THR_EVENT_INT_CLR	RMT_CH1_TX_THR_EVENT_INT_CLR	RMT_CH0_TX_THR_EVENT_INT_CLR	RMT_CH7_ERR_INT_CLR	RMT_CH7_RX_END_INT_CLR	RMT_CH7_TX_END_INT_CLR	RMT_CH6_ERR_INT_CLR	RMT_CH6_RX_END_INT_CLR	RMT_CH6_TX_END_INT_CLR	RMT_CH5_ERR_INT_CLR	RMT_CH5_RX_END_INT_CLR	RMT_CH5_TX_END_INT_CLR	RMT_CH4_ERR_INT_CLR	RMT_CH4_RX_END_INT_CLR	RMT_CH4_TX_END_INT_CLR	RMT_CH3_ERR_INT_CLR	RMT_CH3_RX_END_INT_CLR	RMT_CH3_TX_END_INT_CLR	RMT_CH2_ERR_INT_CLR	RMT_CH2_RX_END_INT_CLR	RMT_CH2_TX_END_INT_CLR	RMT_CH1_ERR_INT_CLR	RMT_CH1_RX_END_INT_CLR	RMT_CH1_TX_END_INT_CLR	RMT_CH0_ERR_INT_CLR	RMT_CH0_RX_END_INT_CLR	RMT_CH0_TX_END_INT_CLR	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

GPIO

- ESP32 has:
  - 56 pins
    - Of these, 45 are user-configurable ("GPIO pins")

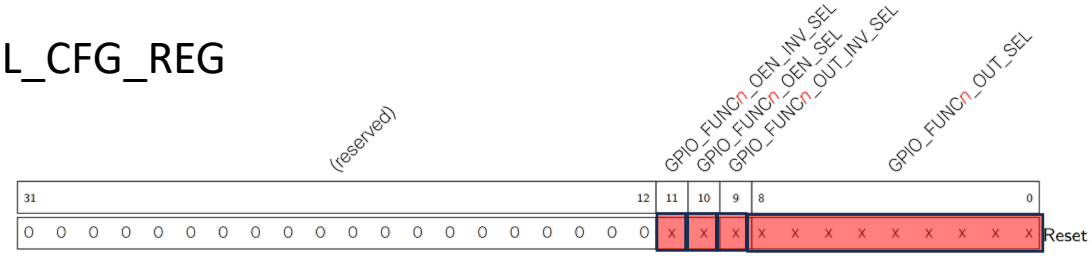
- ESP32 has a 184 x 49 "GPIO matrix" that allows:
  - Any of the ESP32's 49 GPIO_in to drive any of drivable internal signals
  - Any of internal signals to drive any of the ESP32's 49 GPIO_out pins

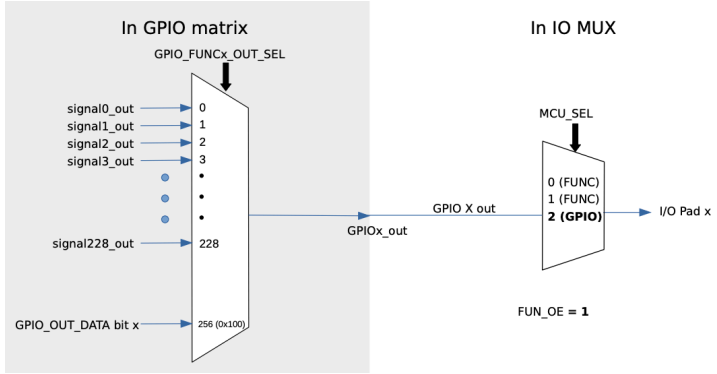
- Each pin is controlled by 2 IO muxes (one for input and one for output)



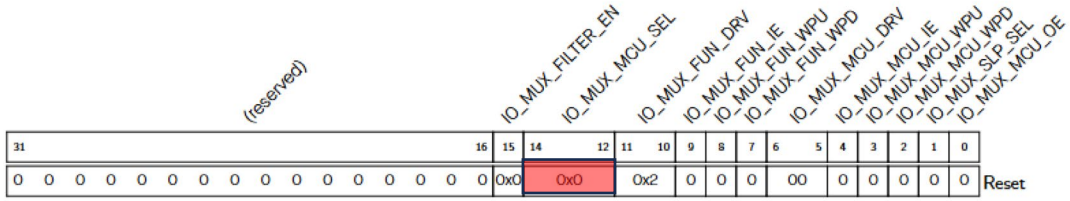
Bit Twiddling

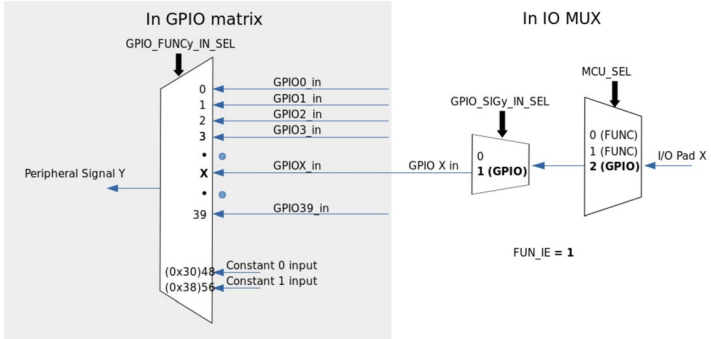
GPIO_FUNCx_OUT_SEL_CFG_REG





IO_MUX_GPIOn_REG





- There is one GPIO_FUNCn_OUT_SEL_CFG_REG register for each of the 49 pins
- There is one GPIO_FUNCn_IN_SEL_CFG_REG register for each of the 49 pins
- There is one IO_MUX_GPIOn_REG for each of the 49 pins

Example

- Goal: connect a “simple” IO output to pin 12
  - Set GPIO_FUNC12_OUT_SEL_CFG_REG.GPIO_FUNC12_OUT_SEL to 256
  - Set IO_MUX_GPIO12_REG.MCU_SEL to 3

```
#define GPIO_BASE                                0x60004000
#define IOMUX_BASE                               0x60009000
#define PIN                                      12
#define GPIO_FUNC##pin##_OUT_SEL_CFG_REG         GPIO_BASE + 0x554 + (pin * 4)
#define IO_MUX_GPIO##pin##_REG                   IOMUX_BASE + (pin * 4)
#define GPIO_OUT_DATA                            GPIO_BASE+4

REG_CLR_BIT(GPIO_FUNC12_OUT_SEL_CFG_REG,(1<<9) - 1);
REG_SET_BIT(GPIO_FUNC12_OUT_SEL_CFG_REG,256);

REG_CLR_BIT(IO_MUX_GPIO12_REG,(1<<3) - 1<<12);
REG_SET_BIT(IO_MUX_GPIO12_REG,3<<12);
REG_SET_BIT(GPIO_OUT_DATA,1<<12);
```

Initializing LEDC

Register 35.7. LEDC_TIMERx_CONF_REG (x: 0-3) (0x00A0+0x8*x)

(reserved)	LEDC_TIMERx_PARA_UP	(reserved)	LEDC_TIMERx_RST	LEDC_TIMERx_PAUSE	LEDC_CLK_DIV_TIMERx	LEDC_TIMERx_DUTY_RES	
31 … 26	25	24	23	22	21 … 4	3 … 0	
0 0 0 0 0 0	0	0	1	0	0x000	0x0	Reset

10 bits . 8 bits

Set this to match audio sample rate

Initializing LEDC

Register 35.1. LEDC_CH*n*_CONF0_REG (*n*: 0-7) (0x0000+0x14n*)**

(reserved)	LEDC_OVF_CNT_RESET_ST_CH*n*	LEDC_OVF_CNT_RESET_CH*n*	LEDC_OVF_CNT_EN_CH*n*	LEDC_OVF_NUM_CH*n*	LEDC_PARA_UP_CH*n*	LEDC_IDLE_LV_CH*n*	LEDC_SIG_OUT_EN_CH*n*	LEDC_TIMER_SEL_CH*n*	
31 … 18	17	16	15	14 … 5	4	3	2	1 … 0	
0 0 0 0 0 0 0 0 0 0 0 0 0 0	0	0	0	0x0	0	0	0	0x0	Reset

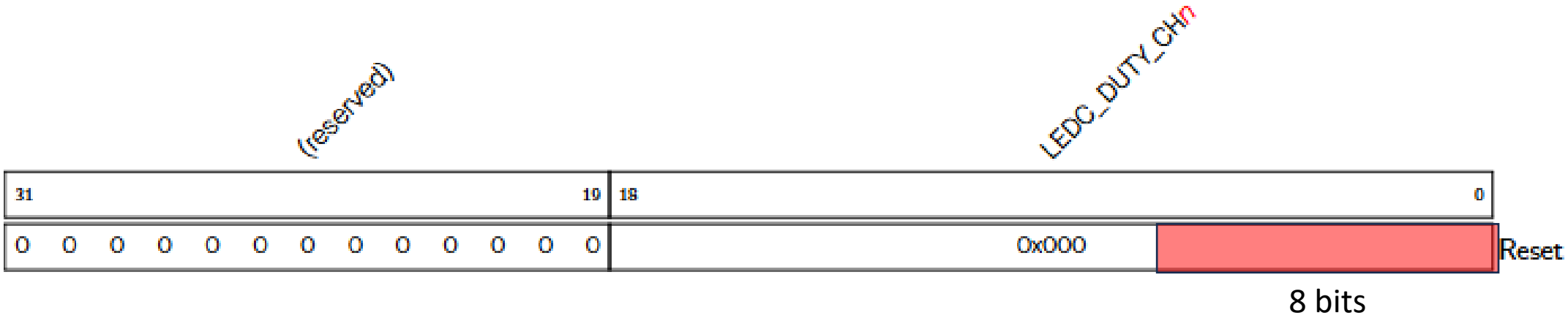
Initlializing LEDC

Register 35.2. LEDC_CH*n*_CONF1_REG (*n*: 0-7) (0x000C+0x14**n*)

LEDC_DUTY_START_CH*n* / LEDC_DUTY_INC_CH*n*		LEDC_DUTY_NUM_CH*n*	LEDC_DUTY_CYCLE_CH*n*	LEDC_DUTY_SCALE_CH*n*	
31	30	29 … 20	19 … 10	9 … 0	
0	1	0x0	0x0	0x0	Reset

Setting Duty Cycle

Register 35.5. LEDC_CH*n*_DUTY_REG (*n*: 0-7) (0x0008+0x14n*)**



LEDC

- To initiate:
  - Set duty cycle (LEDC_CH0_DUTY_REG)
  - Wait until LEDC_HSTIMER0_OVF_INT_RAW is set
  - Clear it by LEDC_HSTIMER0_OVF_INT_CLR

- To convert WAV file:
  - audioToPCM.py <file.wav>
  - generates array.h with:
    - const uint32_t sampleRate = XXX;
    - const uint8_t sampleArray[] = {XXX, XXX, ..., XXX};